

A Drift-Aware MLOps Framework for Multi-Model Serving: A Network Intrusion Detection Case Study

Hoang Tran-Nguyen-Viet¹, Thai Bui-Ngoc¹, Tuan Le-Anh¹, Bao Nguyen-Van¹,
and Quan Le-Trung¹

Faculty of Computer Networks and Communications,
University of Information Technology, VNU-HCM,
Ho Chi Minh City, Vietnam
{23520541,23521412}@gm.uit.edu.vn, {tuanla,baonv,quanlt}@uit.edu.vn

Abstract. Input drift does not directly establish predictive degradation. We present a drift-aware MLOps prototype whose Native Registry links tenant-owned model versions, serving, inference logs, references, and monitoring reports. Its contribution separates distribution alerts, labeled performance, and human-reviewed lifecycle actions; it proposes no new classifier or detector. A controlled NIDS experiment fixes a BENIGN/DDoS XGBoost model across disjoint pools, 120 windows, two sizes, and four alert thresholds. Changing attack prevalence produces strong drift while macro-F1 remains near one. Replacing one quarter of attack samples with held-out attack families reduces mean attack recall to 0.7533 for 1,000-sample windows, although a corrected KS monitor alerts in only one of five windows at a drift-share threshold of 0.30. A four-window service replay of the same model further shows distribution drift without recall loss and recall loss without a live alert. These controlled results do not establish natural drift, retraining effectiveness, or production-scale serving.

Keywords: MLOps · Data Drift · Multi-model Serving · Network Intrusion Detection

1 Introduction

Network intrusion detection encounters changing traffic and attack families, while labels needed to measure predictive quality may arrive late. Generalization beyond the training environment is a longstanding concern [12]. An input-distribution change does not by itself establish predictive degradation; an absent alert does not certify continued accuracy.

MLOps coordinates models, data, deployment, and monitoring [1]. Our question is how to associate evidence with a tenant and model version while keeping update decisions explicit. A Native Registry and replaceable adapters implement this separation. This is a design choice, not a new learning algorithm or a capability claimed to be absent from existing platforms.

We contribute a tenant/model lifecycle design separating evidence from reviewed actions, and a fixed-model NIDS evaluation with stable controls and threshold/window-size sensitivity. We ask: *under which constructed changes does drift agree or disagree with degradation?* Multi-model serving is the architectural scope; the experiment evaluates one classifier, not multi-tenant scalability.

2 Related Work

MLOps architectures cover integrated components [1]. Technical debt studies detail data dependencies and system evolution [2].

TFX integrates validation, training, and serving [3]; MLflow supports tracking and deployment interfaces [4]; Clipper studies online serving [5]. Recent work studies adaptive MLOps [6]. We use existing tools as adapters, without a cross-platform benchmark.

Gama et al. survey concept-drift adaptation [7]; Webb et al. characterize drift types [8]. Continual-learning NIDS evaluates model updates [9]; our fixed-model study measures input shift and labeled performance separately, without identifying concept drift or testing adaptation.

3 Framework and Monitoring Contract

State ownership and execution. The Django control plane stores tenant ownership, model versions, artifact references, readiness metadata, and drift results in PostgreSQL. FastAPI resolves a model record and invokes a loader. Endpoint metadata alone does not prove runtime readiness. Imported artifacts and training outputs share this boundary. MLflow, object storage, and workflow engines are supporting adapters, not owners of product state.

Events carry tenant/model identifiers, features, predictions, timestamps, and optional confidence. Redpanda and a consumer separate persistence from synchronous inference; Evidently provides monitoring. The offline experiment does not establish delivery completeness, failure recovery, or correctness under concurrent updates.

Distribution evidence. For tenant u and deployable model version i , let $\hat{y} = f_{u,i}(x)$, with reference $D_{\text{ref}}^{u,i}$ and current window $D_{\text{cur}}^{u,i}$. A report must retain this model/reference/window association to make the evidence interpretable. For d numeric features, a two-sample test produces statistics δ_j and p-values p_j . In the experiment, Benjamini–Hochberg (BH) adjustment [13] yields q_j within each window, and

$$I_j = \mathbf{1}[q_j < 0.05], \quad \rho = \frac{1}{d} \sum_{j=1}^d I_j, \quad A = \mathbf{1}[\rho \geq \tau_D]. \quad (1)$$

Here ρ is a proportion of flagged features, not a probability of model failure. Marginal tests need not detect changes in feature dependence. BH is a defined

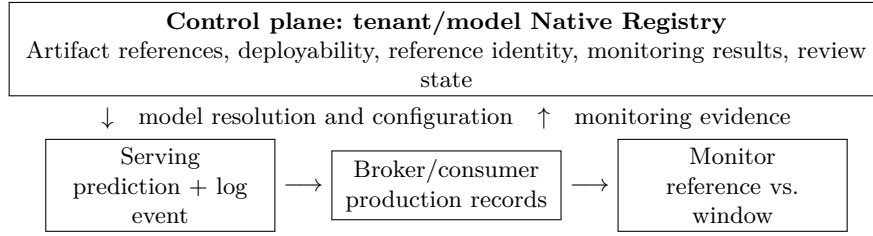


Fig. 1. Logical responsibilities. A distribution alert feeds review; it does not authorize an automatic model replacement. The experiment in Section 4 isolates the monitoring comparison from this deployed pipeline.

comparison rule here, not a guarantee of dataset-level false-alert control under dependent, tied network-flow features.

Review boundary. We distinguish readiness, monitoring evidence (window, settings, scores, alert), and reviewed actions. An alert can request inspection while a model remains deployed. With labels, macro-F1, attack recall, and benign false-positive rate (FPR) provide separate evidence. We propose reviewing shifts without measured harm, and escalating measured harm even when $A = 0$. Retraining would require representative labeled data; promotion would additionally require a challenger to pass held-out quality and operational gates with human approval. These are proposed decision conditions, not an evaluated retraining policy or verified state machine.

4 Controlled NIDS Experiment

4.1 Provenance and Disjoint Pools

CICIDS2017 provides labeled network flows [11]. Our input is a team-subsampled derivative of the third-party *CICIDS2017: Cleaned & Preprocessed* dataset, followed by the team’s *MLOps NIDS Dataset*.¹ The local extraction script caps each label, reserves test rows, and shuffles the remaining rows. The experimental input, named `reference_data.csv` by that earlier pipeline, contains 209,865 rows: 100,000 BENIGN, 50,000 DDoS, 50,000 PortScan, 8,150 BruteForce, and 1,715 WebAttacks, with 52 numeric features.

The associated notebook combines data, cleans values and features, groups labels, and selects features before our split. The local artifact lacks time or capture identifiers; we claim neither a temporal split nor removal of upstream feature-selection leakage. Complete byte-level derivation from the original release remains unverified.

¹ Kaggle source: <https://www.kaggle.com/datasets/ericanacletoribeiro/cicids2017-cleaned-and-preprocessed>; team derivative: <https://www.kaggle.com/datasets/hongtrnnguyenvit/nids-reference-data>.

Table 1. Controlled scenarios; S2 changes attack composition at fixed prevalence.

Scenario	Total attack fraction	Held-out share of attack rows
S0	30%	0% (all DDoS)
S1	15%, 50%, 85%	0% (all DDoS)
S2	30%	25%, 50%, 100%

We hash ordered numeric feature vectors, excluding labels and normalizing signed zero. Twenty rows in ten feature-identical, conflicting-label groups and one further duplicate are removed; there are no nonfinite rows. The remaining 209,844 rows are stratified into train/development/reference/evaluation pools at 50/10/20/20% with split seed 260926. All six pool pairs have zero exact feature-vector overlap, including benign rows. This does not exclude session-level or correlated-flow dependence. The original input files remain unchanged.

4.2 Fixed Model and Window Construction

One binary XGBoost classifier [10] is fitted on 74,994 BENIGN/DDoS rows from the train pool. PortScan, BruteForce, and WebAttacks are deliberately unseen during fitting. We use 200 trees, depth 6, learning rate 0.1, full row/column sampling, the histogram tree method, model seed 42, and prediction threshold 0.5. No scaler, SMOTE, or evaluation-based tuning is applied. Two development windows check execution only; the model remains fixed throughout evaluation.

A separate 5,000-row reference contains 70% BENIGN and 30% DDoS (reference sampling seed 260927). Evaluation windows contain $n \in \{500, 1000\}$ rows sampled without replacement within each window. Table 1 specifies the scenarios. In S2, replacement attacks are divided as evenly as possible among the three held-out families. Thus S2 at 25% replacement and $n = 1000$ contains 700 benign, 225 DDoS, and 75 held-out-family rows. Replacement percentage is a fraction of *attack rows*, not of the entire window.

Each scenario/size uses seeds 42, 123, 456, 789, and 1024. S0 adds seeds 2000–2024, giving 30 stable windows per size and 120 windows overall. Windows may reuse rows from the same evaluation pool; standard deviations describe sampling with one model and one reference, not independent captures or training runs. Class counts are fixed by construction, including in S0.

4.3 Detection, Metrics, and Reproducibility

Two-sided asymptotic Kolmogorov–Smirnov tests cover all 52 features (6,240 tests), followed by Eq. 1. We compare thresholds 0.10, 0.30, 0.50, and 0.70, retaining 0.30 as the principal setting and uncorrected tests as a comparison. Each window yields macro-F1, attack recall, benign FPR, ROC-AUC, family recall, and KS statistics. No threshold is selected retrospectively.

Table 2. Fixed-model results for 1,000-row windows: mean $\pm$ sample SD over five seeds. Alerts use $\tau_D = 0.30$; S0 alert count uses all 30 stable windows.

Scenario	Drift share	Macro-F1	Attack recall	Alerts
S0	0.000 $\pm$ 0.000	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	0/30
S1 (15%)	0.742 $\pm$ 0.059	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	5/5
S1 (50%)	0.850 $\pm$ 0.077	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	5/5
S1 (85%)	0.981 $\pm$ 0.000	0.999 $\pm$ 0.001	1.000 $\pm$ 0.001	5/5
S2 (25%)	0.165 $\pm$ 0.202	0.905 $\pm$ 0.002	0.753 $\pm$ 0.004	1/5
S2 (50%)	0.865 $\pm$ 0.073	0.789 $\pm$ 0.003	0.508 $\pm$ 0.005	5/5
S2 (100%)	0.981 $\pm$ 0.000	0.422 $\pm$ 0.007	0.010 $\pm$ 0.007	5/5

The artifact records settings, versions, checksums, exclusions, pool/window membership, tests, and metrics. A separate verifier reloads the model, recomputes predictions and all KS tests, and checks BH against SciPy; all checks passed. Versions include XGBoost 3.2.0, SciPy 1.17.1, and scikit-learn 1.8.0. Evidently’s deployed preset does not explicitly configure this KS/BH rule, so the checks do not validate that adapter. Code and frozen results are in the paper branch.² Reproduction requires the checksummed local input; public versioned acquisition remains to be finalized.

5 Results

5.1 Distribution Drift and Predictive Quality

Table 2 reports the five principal windows per scenario at $n = 1000$. S1 changes prevalence while retaining the learned attack family. Despite drift shares of 0.7423–0.9808, macro-F1 remains at least 0.9992. These alerts correctly reflect changed input composition; unchanged predictive quality does not make them false distribution alerts.

S2 instead tests held-out attack families. At 25% replacement, attack recall falls to 0.7533, while the principal detector alerts in only 1/5 windows. The same 1/5 count occurs at $n = 500$, where mean recall is 0.7520. Hence low alert frequency can coexist with a material failure to recognize attacks in this constructed setting. At full replacement, recall is 0.0100 for $n = 1000$: PortScan and WebAttacks recall are zero and BruteForce recall averages 0.0300. This is a limitation of the deliberately restricted BENIGN/DDoS baseline, not an evaluation of a model trained on all families.

The S2 trend must be interpreted as a mixture effect: for replacement fraction r , aggregate attack recall equals $(1 - r)R_{\text{DDoS}} + rR_{\text{held-out}}$. Post-hoc inspection of the full evaluation pool finds 9,999/10,000 DDoS, 4/9,998 PortScan, 60/1,630 BruteForce, and 0/344 WebAttacks predicted as attacks. Increasing poorly recognized families therefore explains the decline; it does not demonstrate loss of

² <https://github.com/Viet-Hoang-2005/MLOps-paas-system/tree/codex/csonet-2026-camera-ready>

Table 3. BH-corrected alert counts across drift-share thresholds. S1 labels give total attack prevalence; S2 labels give the fraction of attack rows replaced by held-out families. S0 uses 30 windows per size; other scenarios use five.

Scenario	$n = 500$				$n = 1000$			
	.10	.30	.50	.70	.10	.30	.50	.70
S0	0/30	0/30	0/30	0/30	0/30	0/30	0/30	0/30
S1 (15%)	5/5	5/5	5/5	3/5	5/5	5/5	5/5	3/5
S1 (50%)	5/5	5/5	5/5	4/5	5/5	5/5	5/5	5/5
S1 (85%)	5/5	5/5	5/5	5/5	5/5	5/5	5/5	5/5
S2 (25%)	1/5	1/5	0/5	0/5	2/5	1/5	1/5	0/5
S2 (50%)	5/5	5/5	4/5	1/5	5/5	5/5	5/5	5/5
S2 (100%)	5/5	5/5	5/5	5/5	5/5	5/5	5/5	5/5

DDoS knowledge over time. This inspection reuses the evaluation pool and is not an independent test.

S1 at 85% prevalence and S2 at full replacement both flag approximately 98% of features, yet their macro-F1 values differ greatly. Drift share thus cannot rank predictive harm in these cases. Across all 30 S0 windows at $n = 1000$, mean macro-F1 is 0.999881, recall 0.999889, and FPR 0.000095; perfect values in the five-window table do not hold for every stable draw. S1 at 85% prevalence has mean benign FPR 0.001333 in the principal windows; the other table rows have zero observed FPR.

5.2 Threshold and Window-Size Sensitivity

Table 3 shows alert counts, with 30 windows for each S0 size and five for each shifted setting. All 60 S0 windows have zero BH alerts across the tested thresholds; this finite, conditional observation is not a zero-false-alarm guarantee. At 25% replacement and $n = 1000$, lowering τ_D from 0.30 to 0.10 increases alerts from 1/5 to 2/5, but does not eliminate missed shifts. At 50% replacement and $\tau_D = 0.70$, alerts increase from 1/5 for $n = 500$ to 5/5 for $n = 1000$.

Correction also changes sensitivity. Without BH, S0 at $n = 1000$ and $\tau_D = 0.10$ produces 3/30 alerts instead of 0/30. Conversely, S2 at 25% replacement and $\tau_D = 0.30$ produces 5/5 alerts instead of 1/5. This trade-off should be reported rather than choosing a test after observing its outcomes. Window size affects statistical power; these non-temporal windows do not measure detection delay.

5.3 Controlled Service Replay

We replayed four seed-42, 1,000-row windows through serving, persistence, and the deployed Evidently preset, using the same fixed artifact/reference but separate model versions. Each window had 1,000 unique records matching frozen 52-feature inputs; labels were withheld until scoring. S0 predictions were audited from the UI after a class-0 persistence bug; S1/S2 used the corrected consumer. At drift-share threshold 0.60, S0 and S1 flagged 1/52 and 43/52 features while attack recall stayed 1.0000. S2 with 50% and 100% held-out attack replacement

flagged 22/52 and 29/52, without alerts, while recall fell to 0.5067 and 0.0100. These live counts are not interchangeable with offline KS/BH alerts. S1 needed 1,042 HTTP attempts for 1,000 records after 42 HTTP 502/503 failures during a worker interruption; the other windows needed 1,000. This one-model pilot establishes neither temporal drift nor adaptation or scalability; the earlier 89-second Locust replay remains insufficient for availability claims.

6 Discussion and Limitations

Implication for lifecycle design. Distribution alerts and labeled performance should remain separate, version-associated records. An absent alert should not suppress review when labels reveal failures. The registry makes this distinction explicit; our contribution is the scoped design and evaluation, not novelty in KS, BH, XGBoost, or registry technology.

Data and statistical validity. The study uses a preprocessed, subsampled CI-CIDS2017 derivative with upstream feature selection and no retained temporal metadata. Exact deduplication does not resolve all leakage or capture dependence. High scores on familiar classes may reflect dataset separability. Five shifted windows per setting, shared evaluation pools, fixed class counts, one model/reference, correlated features, and tied-value asymptotic tests limit inference. The construction measures changed composition and observed degradation; it does not isolate a causal mechanism or establish concept drift in $P(Y | X)$.

Security and operational scope. Tenant/model ownership checks address access to records, but do not sandbox model code. A malicious uploaded artifact can execute code through unsafe deserialization, dependency loading, or execution; containerization alone is not evidence of isolation. Restricted artifact formats, isolated execution identities, network/resource limits, and adversarial cross-tenant tests are required before accepting untrusted workloads. No such isolation evaluation is claimed here. Likewise, monitoring completeness under broker failures, model-cache behavior, multi-model/tenant load sweeps, alternative serving platforms, lifecycle ablations, and promotion/rollback gates remain unevaluated.

Further work needs temporal data, other environments, repeated model fits and reference samples, and retraining comparisons. The results support review, not a production threshold or continuous-training benefit.

7 Conclusion

A Native Registry links tenant/model serving and monitoring evidence. Controlled NIDS shifts show both strong drift without substantial degradation and declining recall with infrequent alerts. Sensitivity results reinforce separating alerts from update decisions. Natural drift, automated adaptation, and scalable secure serving require further evidence.

Acknowledgments. This research was supported by the VNUHCM-University of Information Technology’s Scientific Research Fund.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Kreuzberger, D., Kühl, N., Hirschl, S.: Machine Learning Operations (MLOps): Overview, Definition, and Architecture. *IEEE Access* **11**, 31866–31879 (2023). <https://doi.org/10.1109/ACCESS.2023.3262138>
2. Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., Dennison, D.: Hidden Technical Debt in Machine Learning Systems. In: *Advances in Neural Information Processing Systems*, vol. 28 (2015)
3. Baylor, D., et al.: TFX: A TensorFlow-Based Production-Scale Machine Learning Platform. In: *KDD*, pp. 1387–1395 (2017). <https://doi.org/10.1145/3097983.3098021>
4. Zaharia, M., et al.: Accelerating the Machine Learning Lifecycle with MLflow. *IEEE Data Engineering Bulletin* **41**(4), 39–45 (2018)
5. Crankshaw, D., Wang, X., Zhou, G., Franklin, M.J., Gonzalez, J.E., Stoica, I.: Clipper: A Low-Latency Online Prediction Serving System. In: *USENIX NSDI*, pp. 613–627 (2017)
6. Bhatt, H., Biswas, S., Rakhunathan, S., Vaidhyathan, K.: HarmonE: A Self-Adaptive Approach to Architecting Sustainable MLOps. arXiv:2505.13693v1 (2025)
7. Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., Bouchachia, A.: A Survey on Concept Drift Adaptation. *ACM Computing Surveys* **46**(4), article 44, 1–37 (2014). <https://doi.org/10.1145/2523813>
8. Webb, G.I., Hyde, R., Cao, H., Nguyen, H.-L., Petitjean, F.: Characterizing Concept Drift. *Data Mining and Knowledge Discovery* **30**(4), 964–994 (2016). <https://doi.org/10.1007/s10618-015-0448-4>
9. Zhang, X., Zhao, R., Jiang, Z., Chen, H., Ding, Y., Ngai, E.C.H., Yang, S.-H.: Continual Learning with Strategic Selection and Forgetting for Network Intrusion Detection. arXiv:2412.16264v4 (2025)
10. Chen, T., Guestrin, C.: XGBoost: A Scalable Tree Boosting System. In: *KDD*, pp. 785–794 (2016). <https://doi.org/10.1145/2939672.2939785>
11. Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A.: Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In: *ICISSP*, pp. 108–116 (2018). <https://doi.org/10.5220/0006639801080116>
12. Sommer, R., Paxson, V.: Outside the Closed World: On Using Machine Learning for Network Intrusion Detection. In: *IEEE Symposium on Security and Privacy*, pp. 305–316 (2010). <https://doi.org/10.1109/SP.2010.25>
13. Benjamini, Y., Hochberg, Y.: Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. *Journal of the Royal Statistical Society: Series B* **57**(1), 289–300 (1995). <https://doi.org/10.1111/j.2517-6161.1995.tb02031.x>